

LAPORAN PRAKTIKUM STRUKTUR DATA
PEKAN 8
IMPLEMENTASI STRUKTUR DATA: ALGORITMA SORTING DAN
VISUALISASI GUI PADA JAVA



Disusun Oleh:
Annisa Layli Ramadhani
2511532024

Dosen Pengampu: Wahyudi. Dr.. S.T.M.T
Asisten Praktikum: Muhammad Galid Avero

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pertanggungjawaban atas praktikum yang telah dilakukan selama proses pembelajaran mata kuliah Struktur Data.

Laporan praktikum ini dibuat untuk membantu memahami konsep-konsep dasar Struktur Data serta penerapannya dalam pemrograman. Materi yang dibahas mencakup implementasi berbagai struktur data dan algoritma yang digunakan untuk menyelesaikan permasalahan dalam pengolahan data secara lebih efektif dan efisien. Dengan adanya praktikum ini, diharapkan mahasiswa dapat meningkatkan kemampuan logika, analisis, dan keterampilan pemrograman secara lebih baik.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penyusunan maupun isi materi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan laporan praktikum berikutnya.

Padang, 4 Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	5
1.1 Latar Belakang	5
1.2 Tujuan	6
1.3 Manfaat	6
BAB II PEMBAHASAN	7
2.1 Dasar Teori	7
2.1.1 Struktur Data	7
2.1.2 Algoritma Sorting	7
2.1.3 Shell Sort.....	7
2.1.4 Quick Sort.....	8
2.1.5 Merge Sort	8
2.1.6 Graphical User Interface (GUI).....	8
2.1.7 Java Swing	9
2.2 Langkah Kerja.....	9
2.2.1 Kode ShellSort_2511532024.....	9
2.2.2 Penjelasan Kode ShellSort_2511532024	10
2.2.3 Kode QuickSort_2511532024.....	13
2.2.4 Penjelasan Kode QuickSort_2511532024	14
2.2.5 Kode MergeSort_2511532024	17
2.2.6 Penjelasan Kode MergeSort_2511532024.....	19
2.2.7 Kode BubbleSortGUI_2511532024	22
2.2.8 Penjelasan Kode BubbleSortGUI_2511532024	27
2.2.9 Kode MergeSortGUI_2511532024.....	32
2.2.10 Penjelasan Kode MergeSortGUI_2511532024	38
2.2.11 Output Kode Program.....	43
BAB III PENUTUP	45
3.1 Kesimpulan	45
DAFTAR PUSTAKA.....	46

TUGAS PRAKTIKUM STRUKTUR DATA	47
4.1 Soal Tugas	47
4.2 Kode program.....	47
4.2.1 Class Lagu_2511532024.....	47
4.2.2 Class Sorting_2511532024.....	47
4.3 Penjelasan.....	52
4.3.1 Class Lagu_2511532024.....	52
4.3.2 Class Sorting_2511532024.....	53
4.4 Output Kode Program	57

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu materi fundamental dalam ilmu komputer yang berperan penting dalam pengelolaan dan pengolahan data secara efisien. Pemilihan struktur data dan algoritma yang tepat dapat meningkatkan kinerja program, terutama ketika menangani data dalam jumlah besar. Salah satu operasi yang sering dilakukan dalam pengolahan data adalah pengurutan (sorting), yaitu proses menyusun data berdasarkan urutan tertentu, baik secara menaik maupun menurun.

Berbagai algoritma pengurutan telah dikembangkan dengan karakteristik dan tingkat efisiensi yang berbeda-beda. Algoritma seperti shell sort, quick sort, dan merge sort memiliki mekanisme kerja yang unik serta kompleksitas waktu yang berbeda sehingga cocok digunakan pada kondisi tertentu. Pemahaman terhadap algoritma-algoritma tersebut sangat penting bagi mahasiswa untuk meningkatkan kemampuan analisis dalam memilih metode pengurutan yang paling sesuai dengan kebutuhan.

Bahasa pemrograman java menyediakan berbagai fasilitas yang mendukung implementasi algoritma dan struktur data secara efektif, termasuk penggunaan antarmuka grafis (Graphical User Interface/GUI) untuk memvisualisasikan proses kerja algoritma. Melalui visualisasi, proses pengurutan data dapat diamati secara lebih jelas sehingga memudahkan pemahaman terhadap setiap langkah yang dilakukan oleh algoritma.

Oleh karena itu, praktikum ini dilaksanakan untuk mempelajari implementasi algoritma shell sort, quick sort, dan merge sort pada bahasa pemrograman java, serta mengembangkan aplikasi GUI yang dapat menampilkan proses pengurutan data secara interaktif. Dengan praktikum ini diharapkan mahasiswa tidak hanya memahami konsep teoritis algoritma pengurutan, tetapi juga mampu mengimplementasikan dan menganalisis kinerja algoritma tersebut dalam penyelesaian permasalahan nyata.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut:

1. Memahami konsep dasar algoritma pengurutan (sorting) dalam struktur data.
2. Mempelajari cara kerja algoritma shell sort, quick sort, dan merge sort.
3. Mengimplementasikan algoritma pengurutan menggunakan bahasa pemrograman java.
4. Menganalisis proses dan hasil pengurutan data menggunakan berbagai metode sorting.
5. Mengembangkan aplikasi berbasis GUI untuk memvisualisasikan proses pengurutan data secara interaktif.

1.3 Manfaat

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut:

1. Menambah pemahaman mengenai konsep struktur data dan algoritma pengurutan.
2. Meningkatkan keterampilan dalam mengimplementasikan algoritma menggunakan bahasa pemrograman java.
3. Membantu memahami perbedaan mekanisme kerja dan karakteristik masing-masing algoritma sorting.
4. Melatih kemampuan analisis dalam memilih algoritma pengurutan yang sesuai dengan kebutuhan.
5. Memberikan pengalaman dalam pengembangan aplikasi GUI sebagai media visualisasi algoritma.

BAB II

PEMBAHASAN

2.1 Dasar Teori

2.1.1 Struktur Data

Struktur data adalah metode atau cara untuk menyimpan, mengelola, dan mengorganisasi data di dalam komputer agar dapat diakses dan diolah secara efisien. Pemilihan struktur data yang tepat sangat penting karena dapat memengaruhi kinerja program, terutama dalam proses pencarian, penyisipan, penghapusan, dan pengurutan data. Dalam pemrograman, struktur data digunakan untuk mempermudah pengelolaan data sehingga penyelesaian masalah dapat dilakukan secara lebih efektif.

2.1.2 Algoritma Sorting

Sorting atau pengurutan adalah proses menyusun sekumpulan data berdasarkan urutan tertentu, seperti urutan menaik (ascending) atau menurun (descending). Pengurutan data bertujuan untuk mempermudah pencarian, pengelompokan, dan analisis data. Terdapat berbagai algoritma sorting yang memiliki karakteristik dan tingkat efisiensi yang berbeda-beda, di antaranya Shell Sort, Quick Sort, dan Merge Sort.

2.1.3 Shell Sort

Shell Sort merupakan pengembangan dari Insertion Sort yang bekerja dengan membandingkan elemen-elemen yang memiliki jarak tertentu (gap). Pada awal proses, nilai gap dibuat cukup besar, kemudian secara bertahap diperkecil hingga menjadi satu. Dengan cara ini, elemen yang berada jauh dari posisi yang seharusnya dapat dipindahkan lebih cepat dibandingkan menggunakan Insertion Sort biasa. Shell Sort memiliki performa yang lebih baik daripada Insertion Sort pada

data berukuran besar karena mampu mengurangi jumlah pergeseran elemen yang diperlukan.

2.1.4 Quick Sort

Quick Sort adalah algoritma pengurutan yang menggunakan metode divide and conquer. Algoritma ini bekerja dengan memilih sebuah elemen sebagai pivot, kemudian membagi data menjadi dua bagian, yaitu elemen yang lebih kecil dari pivot dan elemen yang lebih besar dari pivot. Setelah proses pembagian selesai, Quick Sort akan memanggil dirinya sendiri secara rekursif untuk mengurutkan kedua bagian tersebut hingga seluruh data terurut. Quick Sort dikenal sebagai salah satu algoritma sorting yang sangat cepat dengan kompleksitas rata-rata $O(n \log n)$.

2.1.5 Merge Sort

Merge Sort juga menggunakan metode divide and conquer. Algoritma ini membagi array menjadi dua bagian yang lebih kecil secara berulang hingga setiap bagian hanya berisi satu elemen. Selanjutnya, bagian-bagian tersebut digabungkan kembali dengan membandingkan elemen-elemen yang ada sehingga menghasilkan urutan yang benar. Merge Sort memiliki kompleksitas waktu $O(n \log n)$ pada kondisi terbaik, rata-rata, maupun terburuk, sehingga cocok digunakan untuk mengolah data dalam jumlah besar.

2.1.6 Graphical User Interface (GUI)

Graphical User Interface (GUI) adalah antarmuka pengguna yang memungkinkan interaksi dengan program melalui komponen visual seperti tombol, kotak teks, label, dan jendela. Dalam bahasa pemrograman Java, GUI dapat dibuat menggunakan pustaka Swing. Penggunaan GUI pada praktikum

ini bertujuan untuk memvisualisasikan proses pengurutan data sehingga pengguna dapat mengamati setiap langkah algoritma secara lebih mudah dan interaktif.

2.1.7 Java Swing

Java Swing merupakan bagian dari Java Foundation Classes (JFC) yang menyediakan berbagai komponen untuk membangun aplikasi desktop berbasis GUI. Komponen seperti JFrame, JPanel, JButton, JTextField, JLabel, dan JTextArea memungkinkan pengembang membuat aplikasi yang interaktif dan mudah digunakan. Pada praktikum ini, Java Swing digunakan untuk menampilkan proses Bubble Sort dan Merge Sort secara langkah demi langkah sehingga memudahkan pemahaman terhadap mekanisme kerja algoritma pengurutan.

2.2 Langkah Kerja

2.2.1 Kode ShellSort_2511532024

```
1. public class ShellSort_2511532024 {
2.
3.     public static void ShellSort_2511532024(int[] A) {
4.         int n_2024 = A.length;
5.         int gap_2024 = n_2024 / 2;
6.         while (gap_2024 > 0) {
7.             for (int i_2024 = gap_2024; i_2024 <
n_2024; i_2024++) {
8.                 int temp_2024 = A[i_2024];
9.                 int j_2024 = i_2024;
10.                while (j_2024 >= gap_2024 && A[j_2024 -
gap_2024] > temp_2024) {
11.                    A[j_2024] = A[j_2024 -
gap_2024];
12.                    j_2024 = j_2024 - gap_2024;
13.                }
14.                A[j_2024] = temp_2024;
15.            }
16.            gap_2024 = gap_2024 / 2;
17.        }
18.    }
19.
20.     public static void main(String[] args) {
21.         int[] data = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
22.         System.out.print("Sebelum: ");
23.         printArray(data);
24.    }
```

```

25.     ShellSort_2511532024(data);
26.
27.     System.out.print("\nSesudah (Shell Sort): ");
28.     printArray(data);
29. }
30.
31. public static void printArray(int[] arr) {
32.     for (int i_2024 : arr)
33.         System.out.print(i_2024 + " ");
34. }

```

2.2.2 Penjelasan Kode ShellSort_2511532024

1. Deklarasi Class dan Method

```

public class ShellSort_2511532024 {
    public static void ShellSort_2024(int arr_2024[]) {

```

Syntax di atas digunakan untuk membuat class dan method Shell Sort. Class berfungsi sebagai tempat penulisan program, sedangkan method ShellSort_2024() digunakan untuk menjalankan proses pengurutan data menggunakan algoritma Shell Sort.

2. Menentukan Panjang Array dan Nilai Gap

```

int n_2024 = arr_2024.length;

for(int gap_2024 = n_2024/2;
    gap_2024 > 0;
    gap_2024 /= 2) {

```

Variabel n_2024 digunakan untuk menyimpan jumlah elemen array. Perulangan for digunakan untuk mengatur nilai gap_2024 yang menjadi jarak antar elemen yang dibandingkan. Nilai gap diawali dengan setengah panjang array dan akan terus diperkecil hingga bernilai satu.

3. Proses Penyisipan Data

```
for(int i_2024 = gap_2024;  
    i_2024 < n_2024;  
    i_2024++) {  
  
    int temp_2024 = arr_2024[i_2024];  
    int j_2024;
```

Syntax di atas digunakan untuk mengambil elemen yang akan diposisikan kembali sesuai urutan. Variabel temp_2024 digunakan sebagai penyimpanan sementara, sedangkan j_2024 digunakan untuk mencari posisi yang tepat bagi elemen tersebut.

4. Proses Perbandingan dan Pergesaran Data

```
for(j_2024 = i_2024;  
    j_2024 >= gap_2024 &&  
    arr_2024[j_2024-gap_2024] > temp_2024;  
    j_2024 -= gap_2024) {  
  
    arr_2024[j_2024] =  
    arr_2024[j_2024-gap_2024];  
}
```

Syntax ini merupakan proses utama Shell Sort. Program membandingkan elemen yang berjarak sesuai nilai gap. Jika elemen sebelumnya lebih besar daripada nilai yang disimpan pada temp_2024, maka elemen tersebut akan digeser ke kanan hingga ditemukan posisi yang sesuai.

5. Menempatkan Data pada Posisi yang Benar

```
arr_2024[j_2024] = temp_2024;
```

Syntax ini digunakan untuk menempatkan nilai yang disimpan pada variabel temp_2024 ke posisi yang sesuai setelah proses pergeseran selesai dilakukan.

6. Method Main dan Deklarasi Data

```
public static void main(String args[]) {  
    int arr_2024[] = {...};  
    int n_2024 = arr_2024.length;
```

Method main() merupakan bagian utama program yang pertama kali dijalankan. Array arr_2024 digunakan untuk menyimpan data yang akan diurutkan, sedangkan variabel n_2024 digunakan untuk menyimpan jumlah elemen array.

7. Menampilkan Data dan Menjalankan Shell Sort

```
System.out.println("Array sebelum sorting:");  
printArray_2024(arr_2024);  
ShellSort_2024(arr_2024);  
System.out.println("Array setelah sorting:");  
printArray_2024(arr_2024);
```

Syntax di atas digunakan untuk menampilkan data sebelum proses pengurutan, memanggil method Shell Sort, dan menampilkan kembali data setelah proses pengurutan selesai dilakukan.

8. Method Print Array

```
public static void printArray_2024(int arr_2024[])
```

Method ini digunakan untuk menampilkan seluruh elemen array ke layar sehingga hasil pengurutan dapat dilihat oleh pengguna.

2.2.3 Kode QuickSort_2511532024

```
1. public class QuickSort_2511532024 {
2.     static void swap_2024(int[] arr, int i_2024, int
   j_2024) {
3.         int temp_2024 = arr[i_2024];
4.         arr[i_2024] = arr[j_2024];
5.         arr[j_2024] = temp_2024;
6.     }
7.     // metode tambahan untuk mengatur pivot menggunakan
   median of three
8.     static void medianOfThree_2024(int[] arr, int
   low_2024, int high_2024) {
9.         int mid_2024 = low_2024 + (high_2024 -
   low_2024) / 2;
10.
11.         // urutkan elemen low, mid, dan high
12.         if (arr[low_2024] > arr[mid_2024]) {
13.             swap_2024(arr, low_2024, mid_2024);
14.         }
15.         if (arr[low_2024] > arr[high_2024]) {
16.             swap_2024(arr, low_2024, high_2024);
17.         }
18.         if (arr[mid_2024] > arr[high_2024]) {
19.             swap_2024(arr, mid_2024, high_2024);
20.         }
21.         swap_2024(arr, mid_2024, high_2024);
22.     }
23.     static int partition_2024(int[] arr, int low_2024,
   int high_2024) {
24.         // panggil fungsi medianOfThree sebelum
   menentukan pivot
25.         medianOfThree_2024(arr, low_2024, high_2024);
26.
27.         int pivot_2024 = arr[high_2024]; // sekarang
   arr[high sudah berisi nilai median
28.         int i_2024 = (low_2024 - 1);
29.
30.         for (int j_2024 = low_2024; j_2024 <=
   high_2024 - 1; j_2024++) {
31.             // jika elemen saat ini lebih kecil
   dari atau sama dengan pivot
32.             if (arr[j_2024] < pivot_2024) {
33.                 // increment indeks elemen yang
   lebih kecil
34.                 i_2024++;
35.                 swap_2024(arr, i_2024, j_2024);
36.             }
37.         }
38.         swap_2024 (arr, i_2024 + 1, high_2024);
39.         return (i_2024 + 1);
40.     }
41.
42.     static void QuickSort_2511532024 (int[] arr, int
   low_2024, int high_2024) {
43.         if (low_2024 < high_2024) {
```

```

44.         int pi_2024 = partition_2024(arr,
45.         low_2024, high_2024);
46.         QuickSort_2511532024(arr, low_2024,
47.         pi_2024 - 1);
48.         QuickSort_2511532024(arr, pi_2024 + 1,
49.         high_2024);
50.     }
51. }
52. public static void printArr_2024(int[] arr) {
53.     for (int i_2024 = 0; i_2024 < arr.length;
54.         i_2024++) {
55.         System.out.print(arr[i_2024] + " ");
56.     }
57.     System.out.println();
58. }
59. public static void main(String[] args) {
60.     int[] arr = { 10, 7, 8, 9, 1, 5 };
61.     int N_2024 = arr.length;
62.     System.out.print("Data sebelum diurutkan: ");
63.     printArr_2024(arr);
64.
65.     QuickSort_2511532024(arr, 0, N_2024 - 1);
66.
67.     System.out.print("Data Terurut QuickSort: ");
68.     printArr_2024(arr);
69. }

```

2.2.4 Penjelasan Kode QuickSort_2511532024

1. Deklarasi Class dan Method

```

public class QuickSort_2511532024 {
    static void swap_2024(int[] arr_2024, int i_2024, int
j_2024) {

```

Syntax di atas digunakan untuk membuat class Quick Sort dan method swap_2024(). Method ini berfungsi untuk menukar posisi dua elemen dalam array selama proses pengurutan berlangsung.

2. Proses Pertukaran Data

```

int temp_2024 = arr_2024[i_2024];
arr_2024[i_2024] = arr_2024[j_2024];
arr_2024[j_2024] = temp_2024;

```

Syntax ini digunakan untuk menukar nilai dua elemen array. Variabel temp_2024 digunakan sebagai tempat penyimpanan sementara agar data tidak hilang saat proses pertukaran dilakukan.

3. Method Median of Three

```
static int medianOfThree_2024(int[] arr_2024, int low_2024,
    int high_2024) {
```

Method medianOfThree_2024() digunakan untuk menentukan nilai pivot yang akan digunakan pada proses partisi. Pivot dipilih dari nilai awal, tengah, dan akhir array sehingga pembagian data menjadi lebih seimbang.

4. Menentukan Nilai Tengah dan Pivot

```
int mid_2024 = low_2024 + (high_2024 - low_2024) / 2;
```

Syntax ini digunakan untuk menentukan indeks tengah array. Nilai tengah tersebut kemudian digunakan bersama elemen awal dan akhir untuk mencari pivot yang paling sesuai.

5. Method Partition

```
static int partition_2024(int[] arr_2024, int low_2024, int
    high_2024) {
```

Method partition_2024() digunakan untuk membagi array menjadi dua bagian berdasarkan nilai pivot. Elemen yang lebih kecil dari pivot akan ditempatkan di sebelah kiri, sedangkan elemen yang lebih besar ditempatkan di sebelah kanan.

6. Proses Perbandingan dan Pemindahan Data

```
for(int j_2024 = low_2024; j_2024 < high_2024; j_2024++)
{
    if(arr_2024[j_2024] < pivot_2024) {
```

Perulangan digunakan untuk membandingkan setiap elemen dengan nilai pivot. Jika elemen lebih kecil dari pivot, maka elemen tersebut dipindahkan ke bagian kiri array melalui proses pertukaran data.

7. Menempatkan Pivot pada Posisi yang Benar

```
swap_2024(arr_2024, i_2024 + 1, high_2024);  
return i_2024 + 1;
```

Syntax ini digunakan untuk menempatkan pivot pada posisi yang seharusnya setelah proses partisi selesai dilakukan. Nilai yang dikembalikan merupakan indeks pivot yang akan digunakan pada proses rekursif berikutnya.

8. Method Quick Sort

```
static void QuickSort_2024(int[] arr_2024, int low_2024, int  
high_2024) {
```

Method ini merupakan proses utama algoritma Quick Sort. Method akan membagi array menjadi beberapa bagian yang lebih kecil, kemudian mengurutkannya secara rekursif hingga seluruh data terurut.

9. Proses Rekursif Quick Sort

```
if(low_2024 < high_2024) {  
    int pi_2024 = partition_2024(arr_2024, low_2024,  
high_2024);  
    QuickSort_2024(arr_2024, low_2024, pi_2024 - 1);  
    QuickSort_2024(arr_2024, pi_2024 + 1, high_2024);  
}
```

Syntax ini digunakan untuk menjalankan proses rekursif pada Quick Sort. Setelah partisi dilakukan, bagian kiri dan kanan pivot akan diurutkan kembali hingga seluruh elemen array berada pada urutan yang benar.

10. Method Main dan Deklarasi Array

```
public static void main(String[] args) {  
    int arr_2024[] = {23, 78, 45, 8, 32, 56, 1};  
    int n_2024 = arr_2024.length;
```

Method main() merupakan bagian utama program yang pertama kali dijalankan. Array arr_2024 digunakan untuk menyimpan data yang akan diurutkan, sedangkan variabel n_2024 digunakan untuk menyimpan jumlah elemen array.

11. Menampilkan Data dan Menjalankan Quick Sort

```
System.out.println("Array sebelum sorting:");  
printArray_2024(arr_2024);  
QuickSort_2024(arr_2024, 0, n_2024 - 1);  
System.out.println("Array setelah sorting:");
```

Syntax di atas digunakan untuk menampilkan data sebelum proses pengurutan, menjalankan algoritma Quick Sort, dan menampilkan kembali data setelah proses pengurutan selesai dilakukan.

12. Method Print Array

```
public static void printArray_2024(int arr_2024[])
```

Method ini digunakan untuk menampilkan seluruh elemen array ke layar sehingga hasil pengurutan dapat dilihat dengan jelas oleh pengguna.

2.2.5 Kode MergeSort_2511532024

```
1. public class MergeSort_2511532024 {  
2. void merge_2024(int arr[], int l_2024, int m_2024,  
   int r_2024 ) {  
3.     // find sizes of two subarrays to be merged  
4.     int n1_2024 = m_2024 - l_2024 + 1;  
5.     int n2_2024 = r_2024 - m_2024;  
6.     /* create temp array */  
7.     int L_2024[] = new int[n1_2024];  
8.     int R_2024[] = new int[n2_2024];  
9.     /* copy data to temp */
```

```

10.     for (int i_2024 = 0; i_2024 < n1_2024;
        ++i_2024)
11.         L_2024[i_2024] = arr[l_2024 + i_2024];
12.     for (int j_2024 = 0; j_2024 < n2_2024;
        ++j_2024 )
13.         R_2024[j_2024] = arr[m_2024 + 1 +
        j_2024];
14.     int i_2024 = 0, j_2024 = 0;
15.     // initial index of merged subarray array
16.     int k_2024 = l_2024;
17.     while (i_2024 < n1_2024 && j_2024 < n2_2024)
    {
18.         if(L_2024[i_2024] <= R_2024[j_2024]) {
19.             arr[k_2024] = L_2024[i_2024];
20.             i_2024++;
21.         } else {
22.             arr[k_2024] = R_2024[j_2024];
23.             j_2024++;
24.         }
25.         k_2024++;
26.     }
27.     /* copy remaining elements of L[] if any */
28.     while (i_2024 < n1_2024) {
29.         arr[k_2024] = L_2024[i_2024];
30.         i_2024++;
31.         k_2024++;
32.     }
33.     /* copy remaining elements of R[] if any */
34.     while (j_2024 < n2_2024) {
35.         arr[k_2024] = R_2024[j_2024];
36.         j_2024++;
37.         k_2024++;
38.     }
39. }
40.
41. void sort_2024 (int arr[], int l_2024, int r_2024) {
42.     if (l_2024 < r_2024) {
43.         // find the middle point
44.         int m_2024 = (l_2024 + r_2024) / 2;
45.         // sort first and second halves
46.         sort_2024(arr, l_2024, m_2024);
47.         sort_2024(arr, m_2024 + 1, r_2024);
48.         // merge the sorted halves
49.         merge_2024(arr, l_2024, m_2024, r_2024);
50.     }
51. }
52. /* A utility function to print array of size n */
53. static void printArray_2024(int arr[]) {
54.     int n_2024 = arr.length;
55.     for(int i_2024 = 0; i_2024 < n_2024; ++i_2024)
56.         System.out.print(arr[i_2024] + " ");
57.     System.out.println();
58. }
59. public static void main(String[] args) {
60.     int arr[] = { 12, 11, 13, 5, 6, 7 };
61.     System.out.print("Sebelum terurut: ");
62.     printArray_2024(arr);

```

```

63.     MergeSort_2511532024 ob = new
        MergeSort_2511532024();
64.     ob.sort_2024(arr, 0, arr.length - 1);
65.     System.out.print("Sesudah Terurut menggunakan
        merge sort: ");
66.     printArray_2024(arr);
67. }
68. }

```

2.2.6 Penjelasan Kode MergeSort_2511532024

1. Deklarasi Class dan Method

```

public class MergeSort_2511532024 {
    void merge_2024(int arr_2024[], int l_2024, int m_2024,
int r_2024) {

```

Syntax di atas digunakan untuk membuat class Merge Sort dan method merge_2024(). Method ini berfungsi untuk menggabungkan dua bagian array yang telah terurut menjadi satu array yang tetap terurut.

2. Menentukan Ukuran Subarray dan Membuat Array Sementara

```

int n1_2024 = m_2024 - l_2024 + 1;
int n2_2024 = r_2024 - m_2024;
int L_2024[] = new int[n1_2024];
int R_2024[] = new int[n2_2024];

```

Syntax di atas digunakan untuk menghitung jumlah elemen pada subarray kiri dan kanan. Setelah itu dibuat array sementara L_2024 dan R_2024 yang digunakan untuk menyimpan data selama proses penggabungan berlangsung.

3. Menyalin Data ke Array Sementara

```

for(int i_2024 = 0; i_2024 < n1_2024; i_2024++)
    L_2024[i_2024] = arr_2024[l_2024 + i_2024];

for(int j_2024 = 0; j_2024 < n2_2024; j_2024++)

```

```
R_2024[j_2024] = arr_2024[m_2024 + 1 + j_2024];
```

Perulangan di atas digunakan untuk menyalin data dari array utama ke array sementara kiri (L_2024) dan kanan (R_2024) sehingga proses penggabungan dapat dilakukan dengan lebih mudah.

4. Proses Penggabungan Data

```
int i_2024 = 0;
int j_2024 = 0;
int k_2024 = l_2024;
while(i_2024 < n1_2024 && j_2024 < n2_2024) {
```

Syntax ini digunakan untuk mempersiapkan proses penggabungan dua subarray yang telah terurut. Variabel i_2024 dan j_2024 digunakan sebagai indeks array sementara, sedangkan k_2024 digunakan sebagai indeks array utama.

5. Perbandingan dan Penyusunan Data

```
if(L_2024[i_2024] <= R_2024[j_2024]) {
    arr_2024[k_2024] = L_2024[i_2024]; i_2024++;
} else {
    arr_2024[k_2024] = R_2024[j_2024]; j_2024++;
}
k_2024++;
```

Syntax ini merupakan proses utama Merge Sort. Program membandingkan elemen dari subarray kiri dan kanan, kemudian memasukkan nilai yang lebih kecil ke dalam array utama hingga seluruh data tersusun secara berurutan.

6. Menyalin Sisa Data

```
while(i_2024 < n1_2024) {
    arr_2024[k_2024] = L_2024[i_2024]; i_2024++;
```

```

k_2024++;
}
while(j_2024 < n2_2024) {
    arr_2024[k_2024] = R_2024[j_2024];    j_2024++;
    k_2024++;
}

```

Syntax di atas digunakan untuk menyalin sisa elemen yang masih terdapat pada subarray kiri atau kanan setelah proses perbandingan selesai dilakukan.

7. Method Sort

```

void sort_2024(int arr_2024[], int l_2024, int r_2024) {

```

Method `sort_2024()` merupakan method utama Merge Sort yang bertugas membagi array menjadi beberapa bagian yang lebih kecil secara rekursif hingga setiap bagian hanya memiliki satu elemen.

8. Proses Pembagian Array Secara Rekursif

```

if(l_2024 < r_2024) {
    int m_2024 = l_2024 + (r_2024 - l_2024) / 2;
    sort_2024(arr_2024, l_2024, m_2024);
    sort_2024(arr_2024, m_2024 + 1, r_2024);
    merge_2024(arr_2024, l_2024, m_2024, r_2024);
}

```

Syntax ini digunakan untuk membagi array menjadi dua bagian yang lebih kecil. Setelah kedua bagian selesai diurutkan secara rekursif, method `merge_2024()` dipanggil untuk menggabungkannya kembali menjadi array yang terurut.

9. Method Main dan Deklarasi Array

```

public static void main(String args[]) {

```

```
int arr_2024[] = {23, 78, 45, 8, 32, 56, 1};  
MergeSort_2511532024 ob_2024 =  
new MergeSort_2511532024();
```

Method main() merupakan bagian utama program yang pertama kali dijalankan. Array arr_2024 digunakan untuk menyimpan data yang akan diurutkan, sedangkan objek ob_2024 digunakan untuk memanggil method Merge Sort.

10. Menampilkan Data dan Menjalankan Merge Sort

```
System.out.println("Array sebelum sorting:");  
printArray_2024(arr_2024);  
ob_2024.sort_2024(arr_2024, 0, arr_2024.length - 1);  
System.out.println("Array setelah sorting:");  
printArray_2024(arr_2024);
```

Syntax di atas digunakan untuk menampilkan data sebelum proses pengurutan, menjalankan algoritma Merge Sort, dan menampilkan kembali data setelah proses pengurutan selesai dilakukan.

11. Method Print Array

```
static void printArray_2024(int arr_2024[])
```

Method printArray_2024() digunakan untuk menampilkan seluruh elemen array ke layar sehingga hasil pengurutan dapat dilihat dengan jelas oleh pengguna.

2.2.7 Kode BubbleSortGUI_2511532024

```
1. public class BubbleSortGUI_2511532024 extends JFrame  
2. {  
3.     private static final long serialVersionUID = 1L;  
4.     private JPanel contentPane_2024;  
5.     private int[] array_2024;  
6.     private JLabel[] labelArray_2024;  
7.     JButton stepButton_2024;  
8.     private JButton resetButton_2024;  
9.     JButton setButton_2024;  
10.    private JTextField inputField_2024;
```

```

11. private JPanel panelArray_2024;
12. private JTextArea stepArea_2024;
13.
14. private int i_2024 = 1, j_2024;
15. private boolean sorting_2024 = false;
16. private int stepCount_2024 = 1;
17.
18. /**
19.  * Create the frame.
20.  */
21. public BubbleSortGUI_2511532024() {
22.
23.     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
24.     setBounds(100, 100, 450, 300);
25.     contentPane_2024 = new JPanel();
26.     contentPane_2024.setBorder(new EmptyBorder(5,
27. 5, 5, 5));
28.     setContentPane(contentPane_2024);
29.     contentPane_2024.setLayout(null);
30.     setTitle("Bubble sort langkah per langkah");
31.     setSize(750, 400);
32.     setLocationRelativeTo(null);
33.     setLayout(new BorderLayout());
34.
35.     // panel input
36.     JPanel inputPanel = new JPanel(new
37. FlowLayout());
38.     inputField_2024 = new JTextField(30);
39.     setButton_2024 = new JButton("Set Array");
40.     inputPanel.add(new JLabel("Masukkan angka
41. (pisahkan dengan koma): "));
42.     inputPanel.add(inputField_2024);
43.     inputPanel.add(setButton_2024);
44.
45.     // panel array visual
46.     panelArray_2024 = new JPanel();
47.     panelArray_2024.setLayout(new FlowLayout());
48.
49.     // panel kontrol
50.     JPanel controlPanel = new JPanel();
51.     stepButton_2024 = new JButton("Langkah
52. Selanjutnya");
53.     resetButton_2024 = new JButton("Reset");
54.     stepButton_2024.setEnabled(false);
55.     controlPanel.add(stepButton_2024);
56.     controlPanel.add(resetButton_2024);
57.
58.     // area teks untuk log langkah-langkah
59.     stepArea_2024 = new JTextArea(8,60);
60.     stepArea_2024.setEditable(false);
61.     stepArea_2024.setFont(new Font("Monospaced",
62. Font.PLAIN, 14));
63.     JScrollPane scrollPane = new
64. JScrollPane(stepArea_2024);
65.
66.     // tambahkan ke panel ke frame
67.     add(inputPanel, BorderLayout.NORTH);

```

```

61.         add(panelArray_2024, BorderLayout.CENTER);
62.         add(controlPanel, BorderLayout.SOUTH);
63.         add(scrollPane, BorderLayout.EAST);
64.
65.         // event set array
66.         setButton_2024.addActionListener(e ->
        setArrayFromInput_2024());
67.
68.         // event langkah selanjutnya
69.         stepButton_2024.addActionListener(e ->
        performStep_2024());
70.
71.         // event reset
72.         resetButton_2024.addActionListener(e ->
        reset_2024());
73.
74. }
75. private void setArrayFromInput_2024() {
76.     String text_2024 =
        inputField_2024.getText().trim();
77.     if (text_2024.isEmpty()) return;
78.     String[] parts_2024 = text_2024.split(",");
79.     array_2024 = new int[parts_2024.length];
80.
81.     try {
82.         for (int k_2024 = 0; k_2024 <
        parts_2024.length; k_2024++) {
83.             array_2024[k_2024] =
        Integer.parseInt(parts_2024[k_2024].trim());
84.         }
85.     } catch (NumberFormatException e_2024) {
86.         JOptionPane.showMessageDialog(this,
        "Masukkan hanya angka " + "yang dipisahkan koma!",
        "Error",
87.             JOptionPane.ERROR_MESSAGE);
88.         return;
89.     }
90.
91.     i_2024 = 0;
92.     j_2024 = 0;
93.     stepCount_2024 = 1;
94.     sorting_2024 = true;
95.
96.     stepButton_2024.setEnabled(true);
97.     stepArea_2024.setText("");
98.     panelArray_2024.removeAll();
99.
100.        labelArray_2024 = new
        JLabel[array_2024.length];
101.        for (int k_2024 = 0; k_2024 <
        array_2024.length; k_2024++) {
102.            labelArray_2024[k_2024] = new
        JLabel(String.valueOf(array_2024[k_2024]));
103.            labelArray_2024[k_2024].setFont( new Font("Arial",
        Font.BOLD, 24));

```



```

104.     labelArray_2024[k_2024].setOpaque(true);
105.     labelArray_2024[k_2024].setBackground(Color.WHITE);
106.     labelArray_2024[k_2024].setBorder(BorderFactory.createLineBorder(Color.BLACK));
107.     labelArray_2024[k_2024].setPreferredSize(new Dimension(50, 50));
108.     labelArray_2024[k_2024].setHorizontalAlignment(Swing Constants.CENTER);
109.     panelArray_2024.add(labelArray_2024[k_2024]);
110.     }
111.
112.         panelArray_2024.revalidate();
113.         panelArray_2024.repaint();
114.     }
115.
116.         private void performStep_2024() {
117.             if (!sorting_2024 || i_2024 >=
array_2024.length - 1) {
118.                 sorting_2024 = false;
119.
stepButton_2024.setEnabled(false);
120.
JOptionPane.showMessageDialog(this, "Sorting
selesai!");
121.                 return;
122.             }
123.
resetHighlights_2024();
124.
StringBuilder stepLog_2024 = new
125.     StringBuilder();
126.
labelArray_2024[j_2024].setBackground(Color.CYAN);
127.
labelArray_2024[j_2024 +
128.     1].setBackground(Color.CYAN);
129.
if (array_2024[j_2024] >
130.     array_2024[j_2024 + 1]) {
131.         // Swap
int temp_2024 =
132.         array_2024[j_2024];
133.         array_2024[j_2024] =
array_2024[j_2024 + 1];
134.         array_2024[j_2024 + 1] =
temp_2024;
135.
labelArray_2024[j_2024].setBackground(Color.RED);
136.
labelArray_2024[j_2024 +
137.     1].setBackground(Color.RED);

```

```

137.         stepLog_2024.append("Langkah
        ").append(stepCount_2024).append(": Menukar elemen
        ke-")
138.         .append(j_2024).append(
        " (").append(array_2024[j_2024 + 1]).append(")
        dengan ke-")
139.         .append(j_2024 +
        1).append("
        (").append(array_2024[j_2024]).append(")\n");
140.     } else {
141.         stepLog_2024.append("Langkah
        ").append(stepCount_2024).append(": Tidak ada
        pertukaran antara ke-")
142.         .append(j_2024).append(
        " dan ke-").append(j_2024 + 1).append("\n");
143.     }
144.
145.         stepLog_2024.append("Hasil:
        ").append(arrayToString_2024(array_2024)).append("\n
        \n");
146.
        stepArea_2024.append(stepLog_2024.toString());
147.         updateLabels_2024();
148.         j_2024++;
149.
150.         if (j_2024 >= array_2024.length -
        i_2024 - 1) {
151.             j_2024 = 0;
152.             i_2024++;
153.         }
154.
155.         stepCount_2024++;
156.
157.         if (i_2024 >= array_2024.length - 1)
        {
158.             sorting_2024 = false;
159.
            stepButton_2024.setEnabled(false);
160.            JOptionPane.showMessageDialog(this, "Sorting
            selesai!");
161.        }
162.    }
163.
164.    private void updateLabels_2024() {
165.        for (int k_2024 = 0; k_2024 <
        array_2024.length; k_2024++) {
166.            labelArray_2024[k_2024].setText(String.valueOf(array
            _2024[k_2024]));
167.        }
168.    }
169.
170.    private void resetHighlights_2024() {
171.        for (JLabel label_2024 :
        labelArray_2024) {

```

```

172.         label_2024.setBackground(Color.WHITE);
173.     }
174. }
175.
176.     private void reset_2024() {
177.         inputField_2024.setText("");
178.         panelArray_2024.removeAll();
179.         panelArray_2024.revalidate();
180.         panelArray_2024.repaint();
181.         stepArea_2024.setText("");
182.         stepButton_2024.setEnabled(false);
183.         sorting_2024 = false;
184.         i_2024 = 0;
185.         j_2024 = 0;
186.         stepCount_2024 = 1;
187.     }
188.
189.     private String arrayToString_2024(int[]
arr_2024) {
190.         StringBuilder sb_2024 = new
StringBuilder();
191.         for (int k_2024 = 0; k_2024 <
arr_2024.length; k_2024++) {
192.             sb_2024.append(arr_2024[k_2024]);
193.             if (k_2024 < arr_2024.length -
1) {
194.                 sb_2024.append(", ");
195.             }
196.         }
197.         return sb_2024.toString();
198.     }
199.     public static void main(String[]
args) {
200.         SwingUtilities.invokeLater(() -> {
201.             BubbleSortGUI_2511532024 gui = new
BubbleSortGUI_2511532024();
202.             gui.setVisible(true);
203.         });
204.     }
205. }

```

2.2.8 Penjelasan Kode BubbleSortGUI_2511532024

1. Deklarasi Class dan Variabel

```

public class BubbleSortGUI_2511532024 extends JFrame {
    private int[] array_2024;
    private JLabel[] labelArray_2024;

```

```
JButton stepButton_2024;  
JButton setButton_2024;  
private JButton resetButton_2024;  
private JTextField inputField_2024;  
private JPanel panelArray_2024;  
private JTextArea stepArea_2024;
```

Syntax di atas digunakan untuk membuat class GUI yang mewarisi `JFrame`. Beberapa variabel digunakan untuk menyimpan data array, label tampilan array, tombol kontrol, area input, dan area log proses Bubble Sort.

2. Constructor dan Pengaturan Frame

```
public BubbleSortGUI_2511532024() {  
    setTitle("Bubble Sort Langkah per Langkah");  
    setSize(750, 400);  
    setLocationRelativeTo(null);  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setLayout(new BorderLayout());  
}
```

Constructor digunakan untuk membuat tampilan utama program GUI. Syntax di atas mengatur judul aplikasi, ukuran jendela, posisi tampilan, layout, dan proses penutupan program.

3. Pembuatan Panel Input

```
JPanel inputPanel = new JPanel(new FlowLayout());  
inputField_2024 = new JTextField(30);  
setButton_2024 = new JButton("Set Array");  
inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma): "));  
inputPanel.add(inputField_2024);  
inputPanel.add(setButton_2024);
```

Panel input digunakan untuk memasukkan data array. Pengguna dapat mengetik beberapa angka yang dipisahkan dengan koma, kemudian menekan tombol Set Array untuk memproses data tersebut.

4. Pembuatan Panel Array dan Tombol Kontrol

```
panelArray_2024 = new JPanel();  
panelArray_2024.setLayout(new FlowLayout());  
stepButton_2024 = new JButton("Langkah Selanjutnya");  
resetButton_2024 = new JButton("Reset");
```

Panel array digunakan untuk menampilkan data secara visual menggunakan label. Tombol Langkah Selanjutnya digunakan untuk menjalankan proses Bubble Sort satu langkah demi satu langkah, sedangkan tombol Reset digunakan untuk mengembalikan program ke kondisi awal.

5. Pembuatan Area Log Proses Sorting

```
stepArea_2024 = new JTextArea(8,60);  
stepArea_2024.setEditable(false);  
JScrollPane scrollPane = new JScrollPane(stepArea_2024);
```

Syntax di atas digunakan untuk membuat area teks yang berfungsi menampilkan setiap langkah proses Bubble Sort. JScrollPane digunakan agar pengguna dapat menggulir isi log apabila jumlah langkah yang ditampilkan cukup banyak.

6. Menambahkan Komponen dan Event Button

```
add(inputPanel, BorderLayout.NORTH);  
add(panelArray_2024, BorderLayout.CENTER);  
add(controlPanel, BorderLayout.SOUTH);  
add(scrollPane, BorderLayout.EAST);  
setButton_2024.addActionListener(  
    e -> setArrayFromInput_2024());
```

```
stepButton_2024.addActionListener(  
    e -> performStep_2024());  
resetButton_2024.addActionListener(  
    e -> reset_2024());
```

Syntax ini digunakan untuk menambahkan seluruh komponen GUI ke dalam frame dan menghubungkan tombol dengan method yang akan dijalankan ketika tombol ditekan.

7. Method Input Array dan Validasi Data

```
String text = inputField_2024.getText().trim();  
String[] parts = text.split(",");  
array_2024 = new int[parts.length];  
for(int k_2024 = 0; k_2024 < parts.length; k_2024++) {  
    array_2024[k_2024] = Integer.parseInt(  
        parts[k_2024].trim());  
}
```

Method ini digunakan untuk mengambil data yang dimasukkan pengguna, memisahkan data berdasarkan tanda koma, kemudian mengubahnya menjadi array bertipe integer yang akan digunakan dalam proses sorting.

8. Menampilkan Array ke GUI

```
labelArray_2024 = new JLabel[array_2024.length];  
for(int k_2024 = 0; k_2024 < array_2024.length;  
    k_2024++) {  
    labelArray_2024[k_2024] = new JLabel(String.valueOf(  
        array_2024[k_2024]));  
    panelArray_2024.add(labelArray_2024[k_2024]);  
}
```

Syntax di atas digunakan untuk membuat tampilan visual array menggunakan komponen JLabel. Setiap elemen array

ditampilkan dalam bentuk kotak yang berisi angka sehingga proses sorting dapat diamati secara langsung.

9. Method Proses Bubble Sort Langkah demi Langkah

```
private void performStep_2024() {  
    if(i_2024 < array_2024.length - 1) {  
        if(array_2024[j_2024] > array_2024[j_2024 + 1]) {
```

Method ini digunakan untuk menjalankan satu langkah proses Bubble Sort setiap kali tombol Langkah Selanjutnya ditekan. Program akan membandingkan dua elemen yang berdekatan untuk menentukan apakah perlu dilakukan pertukaran data.

10. Proses Pertukaran Data

```
int temp_2024 = array_2024[j_2024];  
array_2024[j_2024] = array_2024[j_2024 + 1];  
array_2024[j_2024 + 1] = temp_2024;
```

Syntax ini digunakan untuk menukar posisi dua elemen array yang tidak berada pada urutan yang benar. Variabel temp_2024 digunakan sebagai tempat penyimpanan sementara agar data tidak hilang selama proses pertukaran berlangsung.

11. Menampilkan Hasil Langkah Sorting

```
updateLabels_2024();  
stepArea_2024.append("Langkah "+ stepCount_2024  
    + " : " + arrayToString_2024(array_2024) + "\n");
```

Syntax di atas digunakan untuk memperbarui tampilan array pada GUI dan menampilkan hasil setiap langkah Bubble Sort ke area log sehingga pengguna dapat melihat perubahan data secara bertahap.

12. Method Update Label dan Reset

```
private void updateLabels_2024() {  
    for(int k_2024 = 0; k_2024 < array_2024.length;  
        k_2024++) {  
        labelArray_2024[k_2024].setText(String.valueOf(  
            array_2024[k_2024]));  
    }  
}
```

Method `updateLabels_2024()` digunakan untuk memperbarui tampilan angka pada GUI setelah proses sorting dilakukan. Sedangkan method `reset_2024()` digunakan untuk menghapus seluruh data dan mengembalikan program ke kondisi awal.

13. Method Array Menjadi String dan Main

```
private String arrayToString_2024(int[] arr_2024)
```

Method ini digunakan untuk mengubah isi array menjadi teks agar dapat ditampilkan pada area log proses sorting.

```
public static void main(String[] args) {  
    SwingUtilities.invokeLater() -> {  
        BubbleSortGUI_2511532024 gui =  
            new BubbleSortGUI_2511532024();  
        gui.setVisible(true);  
    });  
}
```

Method `main()` digunakan untuk menjalankan program GUI. `SwingUtilities.invokeLater()` digunakan agar tampilan GUI dijalankan pada thread Swing secara aman dan sesuai dengan standar pemrograman Java.

2.2.9 Kode MergeSortGUI_2511532024

```
1. public class MergeSortGUI_2511532024 extends JFrame  
    {
```



```

2.
3. private static final long serialVersionUID = 1L;
4. private JPanel contentPane_2024;
5. private int[] array_2024;
6. private JLabel[] labelArray_2024;
7. JButton stepButton_2024;
8. private JButton resetButton_2024;
9. JButton setButton_2024;
10. private JTextField inputField_2024;
11. private JPanel panelArray_2024;
12. private JTextArea stepArea_2024;
13.
14. private int i_2024 = 1, j_2024;
15. private boolean sorting_2024 = false;
16. private int stepCount_2024 = 1;
17. private Queue<int[]> mergeQueue_2024 = new
    LinkedList<>();
18.
19. private boolean isMerging_2024 = false;
20. private boolean copying_2024 = false;
21. private int left_2024;
22. private int mid_2024;
23. private int right_2024;
24. private int k_2024;
25. private int[] temp_2024;
26.
27. /**
28.  * Create the frame.
29.  * @return
30.  */
31. public MergeSortGUI_2511532024() {
32.
33.     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
34.     setBounds(100, 100, 450, 300);
35.     contentPane_2024 = new JPanel();
36.     contentPane_2024.setBorder(new EmptyBorder(5,
37. 5, 5, 5));
38.     setContentPane(contentPane_2024);
39.     contentPane_2024.setLayout(null);
40.     setTitle("Merge sort langkah per langkah");
41.     setSize(750, 400);
42.     setLocationRelativeTo(null);
43.     setLayout(new BorderLayout());
44.
45.     // panel input
46.     JPanel inputPanel = new JPanel(new
47.     FlowLayout());
48.     inputField_2024 = new JTextField(30);
49.     setButton_2024 = new JButton ("Set Array");
50.     inputPanel.add(new JLabel("Masukkan angka
51. (pisahkan dengan koma): "));
52.     inputPanel.add(inputField_2024);
53.     inputPanel.add(setButton_2024);
54.
55.     // panel array visual
56.     panelArray_2024 = new JPanel();
57.     panelArray_2024.setLayout(new FlowLayout());

```

```

54.
55.     // panel kontrol
56.     JPanel controlPanel = new JPanel();
57.     stepButton_2024 = new JButton("Langkah
Selanjutnya");
58.     resetButton_2024 = new JButton("Reset");
59.     stepButton_2024.setEnabled(false);
60.     controlPanel.add(stepButton_2024);
61.     controlPanel.add(resetButton_2024);
62.
63.     // area teks untuk log langkah-langkah
64.     stepArea_2024 = new JTextArea(8,60);
65.     stepArea_2024.setEditable(false);
66.     stepArea_2024.setFont(new Font("Monospaced",
Font.PLAIN, 14));
67.     JScrollPane scrollPane = new
JScrollPane(stepArea_2024);
68.
69.     // tambahkan ke panel ke frame
70.     add(inputPanel, BorderLayout.NORTH);
71.     add(panelArray_2024, BorderLayout.CENTER);
72.     add(controlPanel, BorderLayout.SOUTH);
73.     add(scrollPane, BorderLayout.EAST);
74.
75.     // event set array
76.     setButton_2024.addActionListener(e ->
setArrayFromInput_2024());
77.
78.     // event langkah selanjutnya
79.     stepButton_2024.addActionListener(e ->
performStep_2024());
80.
81.     // event reset
82.     resetButton_2024.addActionListener(e ->
reset_2024());
83.
84. }
85.
86. private void setArrayFromInput_2024() {
87.
88.     String text_2024 =
inputField_2024.getText().trim();
89.     if (text_2024.isEmpty()) return;
90.     String[] parts_2024 = text_2024.split(",");
91.     array_2024 = new int[parts_2024.length];
92.
93.     try {
94.         for (int i_2024 = 0; i_2024 <
parts_2024.length; i_2024++) {
95.             array_2024[i_2024] =
Integer.parseInt(parts_2024[i_2024].trim());
96.         }
97.     } catch (NumberFormatException e_2024) {
98.         JOptionPane.showMessageDialog(this,
"Masukkan hanya angka!", "Error",
JOptionPane.ERROR_MESSAGE);
99.         return;

```

```

100.         }
101.
102.         labelArray_2024 = new
JLabel[array_2024.length];
103.         panelArray_2024.removeAll();
104.
105.         for (int i_2024 = 0; i_2024 <
array_2024.length; i_2024++) {
106.             labelArray_2024[i_2024] = new
JLabel(String.valueOf(array_2024[i_2024]));
107.             labelArray_2024[i_2024].setFont(new Font("Arial",
Font.BOLD, 24));
108.             labelArray_2024[i_2024].setOpaque(true);
109.             labelArray_2024[i_2024].setBackground(Color.WHITE);
110.             labelArray_2024[i_2024].setBorder(BorderFactory.crea
teLineBorder(Color.BLACK));
111.             labelArray_2024[i_2024].setPreferredSize(new
Dimension(50, 50));
112.             labelArray_2024[i_2024].setHorizontalAlignment(Swing
Constants.CENTER);
113.             panelArray_2024.add(labelArray_2024[i_2024]);
114.
115.             mergeQueue_2024.clear();
116.             generateMergeSteps_2024(0,
array_2024.length - 1);
117.
118.             stepButton_2024.setEnabled(true);
119.             stepArea_2024.setText("");
120.
121.             stepCount_2024 = 1;
122.             isMerging_2024 = false;
123.
124.             panelArray_2024.revalidate();
125.             panelArray_2024.repaint();
126.         }
127.     }
128.     private void
generateMergeSteps_2024(int left_2024, int
right_2024) {
129.         if (left_2024 >= right_2024) {
130.             return;
131.         }
132.         int mid_2024 = (left_2024 +
right_2024) / 2;
133.         generateMergeSteps_2024(left_2024, mid_2024);
134.         generateMergeSteps_2024(mid_2024 + 1, right_2024);

```

```

135.             mergeQueue_2024.offer(new int[]
136.             {
137.                 left_2024, mid_2024,
138.                 right_2024
139.             });
140.         }
141.         private void performStep_2024() {
142.             resetHighlights_2024();
143.             if (!isMerging_2024
144.                 && !mergeQueue_2024.isEmpty()) {
145.                 int[] range_2024 =
146.                     mergeQueue_2024.poll();
147.                 left_2024 = range_2024[0];
148.                 mid_2024 = range_2024[1];
149.                 right_2024 = range_2024[2];
150.                 temp_2024 = new int[right_2024
151.                     - left_2024 + 1];
152.                 i_2024 = left_2024;
153.                 j_2024 = mid_2024 + 1;
154.                 k_2024 = 0;
155.                 copying_2024 = false;
156.                 isMerging_2024 = true;
157.                 stepArea_2024.append("Langkah "
158.                     + stepCount_2024++ + ": Mulai merge dari " +
159.                     left_2024 + " ke "
160.                     + right_2024 + "\n");
161.                 return;
162.             }
163.             if (isMerging_2024 && !copying_2024)
164.             {
165.                 if (i_2024 <= mid_2024 &&
166.                     j_2024 <= right_2024) {
167.                     labelArray_2024[i_2024].setBackground(Color.CYAN);
168.                     labelArray_2024[j_2024].setBackground(Color.CYAN);
169.                     if (array_2024[i_2024] <=
170.                         array_2024[j_2024]) {
171.                         temp_2024[k_2024++] =
172.                             array_2024[i_2024++];
173.                     } else {
174.                         temp_2024[k_2024++] =
175.                             array_2024[j_2024++];
176.                     }
177.                 }
178.                 stepArea_2024.append("Langkah " + stepCount_2024++ +
179.                     ": Bandingkan dan salin elemen\n");
180.                 return;
181.             } else if (i_2024 <= mid_2024)
182.             {
183.                 temp_2024[k_2024++] =
184.                     array_2024[i_2024++];

```

```

172.         stepArea_2024.append("Langkah " + stepCount_2024++ +
173.             ": Salin sisa kiri\n");
174.         return;
175.     } else if (j_2024 <= right_2024)
176.     {
177.         temp_2024[k_2024++] =
178.         array_2024[j_2024++];
179.         stepArea_2024.append("Langkah " + stepCount_2024++ +
180.             ": Salin sisa kanan\n");
181.         return;
182.     } else {
183.         copying_2024 = true;
184.         k_2024 = 0;
185.         return;
186.     }
187. }
188.
189. if (copying_2024 && k_2024 <
190.     temp_2024.length) {
191.     array_2024[left_2024 + k_2024]
192.     = temp_2024[k_2024];
193.     labelArray_2024[left_2024 +
194.         k_2024].setText(String.valueOf(temp_2024[k_2024]));
195.     labelArray_2024[left_2024 +
196.         k_2024].setBackground(Color.GREEN);
197.     k_2024++;
198.     stepArea_2024.append("Langkah "
199.         + stepCount_2024++ + ": Tempelkan ke array utama\n");
200.     return;
201. }
202.
203. if (copying_2024 && k_2024 ==
204.     temp_2024.length) {
205.     isMerging_2024 = false;
206.     copying_2024 = false;
207. }
208.
209. if (mergeQueue_2024.isEmpty()
210.     && !isMerging_2024) {
211.     stepArea_2024.append("Selesai.\n");
212.     stepButton_2024.setEnabled(false);
213.     JOptionPane.showMessageDialog(this, "Merge Sort
214.         selesai!");
215.     }
216. }
217.
218. private void resetHighlights_2024()
219. {
220.     if (labelArray_2024 == null)
221.     return;
222.     for (JLabel label_2024 :
223.         labelArray_2024) {

```

```

209.         label_2024.setBackground(Color.WHITE);
210.     }
211. }
212.
213.     private void reset_2024() {
214.         inputField_2024.setText("");
215.         panelArray_2024.removeAll();
216.         panelArray_2024.revalidate();
217.         panelArray_2024.repaint();
218.         stepArea_2024.setText("");
219.
220.         stepButton_2024.setEnabled(false);
221.         mergeQueue_2024.clear();
222.         isMerging_2024 = false;
223.         stepCount_2024 = 1;
224.     }
225.     public static void main(String[]
args) {
226.         SwingUtilities.invokeLater(() -> {
227.             MergeSortGUI_2511532024 gui = new
MergeSortGUI_2511532024();
228.             gui.setVisible(true);
229.         });
230.     }

```

2.2.10 Penjelasan Kode MergeSortGUI_2511532024

1. Deklarasi Class dan Variabel

```

public class MergeSortGUI_2511532024 extends JFrame {
    private int[] array_2024;
    private JLabel[] labelArray_2024;
    private JTextField inputField_2024;
    private JButton setButton_2024;
    private JButton nextButton_2024;
    private JButton resetButton_2024;
    private JTextArea prosesArea_2024;
    private JPanel arrayPanel_2024;

```

Syntax di atas digunakan untuk membuat class GUI yang mewarisi JFrame. Variabel yang dideklarasikan digunakan untuk menyimpan data array, menampilkan elemen array,

menerima input pengguna, serta mengontrol proses Merge Sort melalui tombol dan area proses.

2. Constructor dan Pengaturan Tampilan

```
public MergeSortGUI_2511532024() {  
    setTitle("Visualisasi Merge Sort");  
    setSize(800,500);  
    setLocationRelativeTo(null);  
    setDefaultCloseOperation(EXIT_ON_CLOSE);  
    setLayout(new BorderLayout());  
}
```

Constructor digunakan untuk membangun tampilan utama aplikasi. Syntax ini mengatur judul jendela, ukuran frame, posisi tampilan di tengah layar, metode penutupan program, dan layout yang digunakan.

3. Pembuatan Komponen Input

```
inputField_2024 = new JTextField(30);  
setButton_2024 = new JButton("Set Array");  
JPanel inputPanel_2024 = new JPanel(new FlowLayout());
```

Syntax ini digunakan untuk membuat area input yang memungkinkan pengguna memasukkan data array serta tombol untuk memproses data yang telah dimasukkan.

4. Pembuatan Tombol Kontrol

```
nextButton_2024 = new JButton("Langkah Berikutnya");  
resetButton_2024 = new JButton("Reset");
```

Tombol Langkah Berikutnya digunakan untuk menjalankan proses Merge Sort secara bertahap, sedangkan tombol Reset digunakan untuk mengembalikan program ke kondisi awal.

5. Pembuatan Area Array dan Log Proses

```
arrayPanel_2024 = new JPanel();
```

```
prosesArea_2024 = new JTextArea();  
JScrollPane scrollPane_2024 =  
    new JScrollPane(prosesArea_2024);
```

Syntax di atas digunakan untuk membuat panel visualisasi array dan area teks yang menampilkan setiap proses penggabungan data selama Merge Sort berlangsung.

6. Menambahkan Komponen ke Frame

```
add(inputPanel_2024, BorderLayout.NORTH);  
add(arrayPanel_2024, BorderLayout.CENTER);  
add(scrollPane_2024, BorderLayout.SOUTH);
```

Syntax ini digunakan untuk menempatkan seluruh komponen GUI ke dalam frame sesuai posisi yang telah ditentukan menggunakan BorderLayout.

7. Event Handling Tombol

```
setButton_2024.addActionListener(  
    e -> setArray_2024());  
nextButton_2024.addActionListener(  
    e -> nextStep_2024());  
resetButton_2024.addActionListener(  
    e -> reset_2024());
```

Syntax di atas digunakan untuk menghubungkan tombol dengan method yang akan dijalankan ketika tombol ditekan oleh pengguna.

8. Membaca Input dan Membuat Array

```
String input_2024 = inputField_2024.getText();  
String[] data_2024 = input_2024.split(",");  
array_2024 = new int[data_2024.length];
```

Syntax ini digunakan untuk membaca data yang dimasukkan pengguna, memisahkan setiap angka berdasarkan tanda

koma, kemudian menyiapkan array integer untuk menyimpan data tersebut.

9. Konversi String ke Integer

```
for(int i_2024 = 0; i_2024 < data_2024.length; i_2024++){  
    array_2024[i_2024] = Integer.parseInt(  
        data_2024[i_2024].trim());  
}
```

Perulangan ini digunakan untuk mengubah setiap data bertipe String menjadi integer sehingga dapat diproses oleh algoritma Merge Sort.

10. Menampilkan Array pada GUI

```
labelArray_2024 = new JLabel[array_2024.length];  
for(int i_2024 = 0; i_2024 < array_2024.length;  
    i_2024++){  
    labelArray_2024[i_2024] = new JLabel(String.valueOf(  
        array_2024[i_2024]));  
}
```

Syntax ini digunakan untuk membuat visualisasi array menggunakan komponen JLabel. Setiap elemen array ditampilkan sehingga pengguna dapat melihat perubahan data selama proses sorting berlangsung.

11. Method Merge Sort

```
void mergeSort_2024(int arr_2024[], int left_2024,  
    int right_2024){
```

Method ini merupakan algoritma utama Merge Sort yang bertugas membagi array menjadi beberapa bagian yang lebih kecil secara rekursif hingga setiap bagian hanya memiliki satu elemen.

12. Proses Pembagian Array

```
if(left_2024 < right_2024){  
    int mid_2024 = (left_2024 + right_2024)/2;  
    mergeSort_2024(arr_2024, left_2024, mid_2024);  
    mergeSort_2024(arr_2024, mid_2024 + 1, right_2024);  
}
```

Syntax ini digunakan untuk membagi array menjadi dua bagian yang lebih kecil. Proses pembagian dilakukan secara rekursif hingga diperoleh subarray yang berukuran satu elemen.

13. Method Merge

```
void merge_2024(int arr_2024[], int left_2024,  
    int mid_2024, int right_2024){
```

Method merge_2024() digunakan untuk menggabungkan dua subarray yang telah terurut menjadi satu array yang tetap berada dalam urutan yang benar.

14. Perbandingan dan Penggabungan Data

```
while(i_2024 < n1_2024 && j_2024 < n2_2024){  
    if(L_2024[i_2024] <= R_2024[j_2024]){
```

Method merge_2024() digunakan untuk menggabungkan dua subarray yang telah terurut menjadi satu array yang tetap berada dalam urutan yang benar.

15. Menampilkan Proses Sorting

```
prosesArea_2024.append("Merge : " + Arrays.toString(  
    array_2024) + "\n");  
updateArray_2024();
```

Syntax ini digunakan untuk menampilkan setiap hasil proses penggabungan ke area log dan memperbarui tampilan array

pada GUI agar pengguna dapat melihat perubahan data secara langsung.

16. Method Reset dan Main

```
private void reset_2024(){
```

Method reset_2024() digunakan untuk menghapus data yang sedang diproses dan mengembalikan tampilan program ke kondisi awal.

```
public static void main(String[] args){  
    SwingUtilities.invokeLater( () -> {  
        new MergeSortGUI_2511532024().setVisible(true);  
    });  
}
```

Method main() merupakan titik awal program. SwingUtilities.invokeLater() digunakan untuk menjalankan GUI pada Event Dispatch Thread sehingga aplikasi dapat berjalan dengan aman dan responsif.

2.2.11 Output Kode Program

1. Output Kode Program ShellSort_2511532024

```
Sebelum: 3 10 4 6 8 9 7 2 1 5  
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

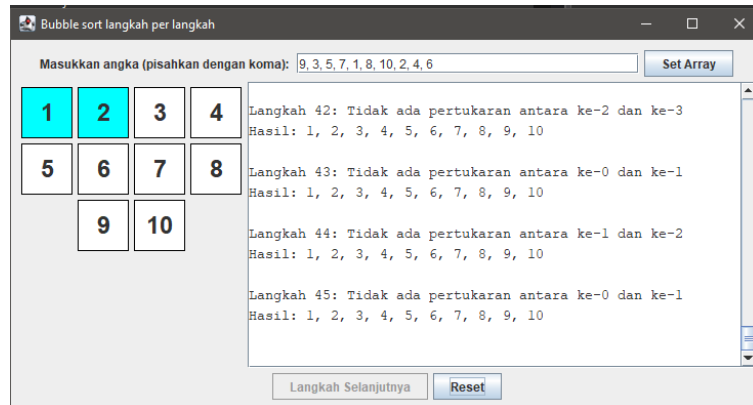
2. Output Kode Program QuickSort_2511532024

```
Data sebelum diurutkan: 10 7 8 9 1 5  
Data Terurut QuickSort: 1 5 7 8 9 10
```

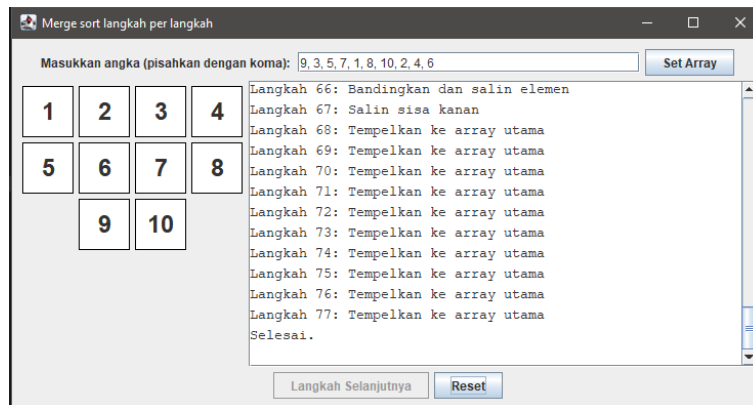
3. Output Kode Program MergeSort_2511532024

```
Sebelum terurut: 12 11 13 5 6 7  
Sesudah Terurut menggunakan merge sort: 5 6 7 11 12 13
```

4. Output Kode Program BubbleSortGUI_2511532024



5. Output Kode Program MergeSortGUI_2511532024



BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma pengurutan (sorting) merupakan salah satu konsep penting dalam struktur data yang digunakan untuk menyusun data agar lebih teratur dan mudah diolah. Pada praktikum ini telah berhasil diimplementasikan beberapa algoritma sorting, yaitu Shell Sort, Quick Sort, dan Merge Sort, menggunakan bahasa pemrograman Java.

Hasil implementasi menunjukkan bahwa setiap algoritma memiliki mekanisme kerja yang berbeda dalam melakukan pengurutan data. Shell Sort melakukan pengurutan dengan memanfaatkan nilai gap tertentu, Quick Sort menggunakan metode divide and conquer dengan pemilihan pivot untuk membagi data, sedangkan Merge Sort membagi data menjadi beberapa bagian kecil kemudian menggabungkannya kembali dalam keadaan terurut. Meskipun memiliki cara kerja yang berbeda, ketiga algoritma tersebut mampu menghasilkan data yang terurut dengan benar.

Selain itu, pembuatan aplikasi berbasis Graphical User Interface (GUI) pada Bubble Sort dan Merge Sort berhasil memberikan visualisasi proses pengurutan secara interaktif. Visualisasi tersebut membantu pengguna memahami tahapan algoritma secara lebih jelas karena setiap proses perbandingan, pertukaran, maupun penggabungan data dapat diamati secara langsung.

Dengan terlaksananya praktikum ini, pemahaman mengenai konsep algoritma pengurutan, implementasi struktur data dalam Java, serta pengembangan aplikasi GUI dapat meningkat sehingga menjadi bekal yang bermanfaat dalam pengembangan perangkat lunak dan penyelesaian permasalahan komputasi di masa mendatang.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Merge Sort," [Online]. Available: <https://www.geeksforgeeks.org/merge-sort/>. [Diakses: 4 Juni 2026].
- [2] GeeksforGeeks, "Quick Sort Algorithm," [Online]. Available: <https://www.geeksforgeeks.org/quick-sort-algorithm/>. [Diakses: 4 Juni 2026].
- [3] GeeksforGeeks, "Shell Sort Algorithm," [Online]. Available: <https://www.geeksforgeeks.org/shellsort/>. [Diakses: 4 Juni 2026].
- [4] Oracle Corporation, "JButton (Java Platform SE Documentation)," [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/javax/swing/JButton.html>. [Diakses: 4 Juni 2026].
- [5] Oracle Corporation, "JFrame (Java Platform SE Documentation)," [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/javax/swing/JFrame.html>. [Diakses: 4 Juni 2026].

TUGAS PRAKTIKUM STRUKTUR DATA

4.1 Soal Tugas

Mahasiswa diminta mengimplementasikan salah SATU algoritma sorting untuk mengurutkan data lagu dalam playlist. Pilih salah satu: Shell Sort, Quick Sort, atau Merge Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (AZ) atau Durasi (terpendek ke terpanjang).

4.2 Kode program

4.2.1 Class Lagu_2511532024

```
1. public class Lagu_2511532024 {
2.     String judul_2024;
3.     String penyanyi_2024;
4.     int durasi_2024;
5.
6.     public Lagu_2511532024(
7.         String judul_2024,
8.         String penyanyi_2024,
9.         int durasi_2024) {
10.
11.         this.judul_2024 = judul_2024;
12.         this.penyanyi_2024 = penyanyi_2024;
13.         this.durasi_2024 = durasi_2024;
14.     }
15. }
```

4.2.2 Class Sorting_2511532024

```
1. public class Sorting_2511532024 {
2.     static Lagu_2511532024[] dataLagu_2024 = new
Lagu_2511532024[20];
3.     static int jumlahData_2024 = 0;
4.
5.     static void inputData_2024() {
6.         dataLagu_2024[jumlahData_2024++] = new
Lagu_2511532024(
7.             "The Man Who Can't Be
Moved",
8.             "The Script",
9.             241);
10.
11.         dataLagu_2024[jumlahData_2024++] = new
Lagu_2511532024(
12.             "The Apartment We Won't
Share",
13.             "NIKI",
14.             150);
15. }
```

```

15.
16.         dataLagu_2024[jumlahData_2024++] = new
Lagu_2511532024(
17.             "Best Friend",
18.             "Rex Orange County",
19.             262);
20.
21.         dataLagu_2024[jumlahData_2024++] = new
Lagu_2511532024(
22.             "Needy",
23.             "Ariana Grande",
24.             172);
25.
26.         dataLagu_2024[jumlahData_2024++] = new
Lagu_2511532024(
27.             "Hold Me Down",
28.             "Daniel Caesar",
29.             231);
30.
31.         dataLagu_2024[jumlahData_2024++] = new
Lagu_2511532024(
32.             "Multo",
33.             "Cup of Joe",
34.             238);
35.
36.         dataLagu_2024[jumlahData_2024++] = new
Lagu_2511532024(
37.             "Bags",
38.             "Clairo",
39.             261);
40.
41.         dataLagu_2024[jumlahData_2024++] = new
Lagu_2511532024(
42.             "Seasons",
43.             "wave to earth",
44.             256);
45.     }
46.
47.     static void tampilData_2024() {
48.         for(int i_2024 = 0; i_2024 <
jumlahData_2024; i_2024++) {
49.             System.out.println((i_2024 + 1)
+ ". " + dataLagu_2024
50.                 [i_2024].judul_2024 + " - " +
dataLagu_2024[i_2024].durasi_2024
51.                 + " detik");
52.         }
53.     }
54.
55.     static void shellSort_2024() {
56.         for(int gap_2024 = jumlahData_2024 / 2;
gap_2024 > 0; gap_2024 /= 2) {
57.             for(int i_2024 = gap_2024;
i_2024 < jumlahData_2024; i_2024++) {
58.                 Lagu_2511532024 temp_2024
= dataLagu_2024[i_2024];
59.

```



```

60.             int j_2024;
61.
62.             for(j_2024 = i_2024;
j_2024 >= gap_2024 && dataLagu_2024
63.                 [j_2024 -
gap_2024].judul_2024.compareToIgnoreCase
64.                 (temp_2024.judul_2024) > 0; j_2024 -=
gap_2024) {
65.
66.                 dataLagu_2024[j_2024] = dataLagu_2024[j_2024
- gap_2024];
67.             }
68.
69.             dataLagu_2024[j_2024] =
temp_2024;
70.
71.         }
72.     }
73. }
74.
75.     static int partition_2024(int low_2024, int
high_2024) {
76.         int pivot_2024 =
dataLagu_2024[high_2024].durasi_2024;
77.         int i_2024 = low_2024 - 1;
78.
79.         for(int j_2024 = low_2024; j_2024 <
high_2024; j_2024++) {
80.
81.             if(dataLagu_2024[j_2024].durasi_2024 <
pivot_2024) {
82.                 i_2024++;
83.
84.                 Lagu_2511532024 temp_2024
= dataLagu_2024[i_2024];
85.                 dataLagu_2024[i_2024] =
dataLagu_2024[j_2024];
86.                 dataLagu_2024[j_2024] =
temp_2024;
87.             }
88.
89.             Lagu_2511532024 temp_2024 =
dataLagu_2024[i_2024 + 1];
90.             dataLagu_2024[i_2024 + 1] =
dataLagu_2024[high_2024];
91.             dataLagu_2024[high_2024] = temp_2024;
92.
93.             return i_2024 + 1;
94.         }
95.
96.     static void quickSort_2024(int low_2024, int
high_2024) {
97.         if(low_2024 < high_2024) {

```

```

98.         int pi_2024 =
partition_2024(low_2024, high_2024);
99.
100.        quickSort_2024(low_2024, pi_2024
- 1);
101.        quickSort_2024(low_2024 + 1,
pi_2024);
102.    }
103. }
104.
105.     static void merge_2024(int left_2024, int
mid_2024, int right_2024) {
106.         int n1_2024 = mid_2024 - left_2024 + 1;
107.         int n2_2024 = right_2024 - mid_2024;
108.
109.         Lagu_2511532024[] L_2024 = new
Lagu_2511532024[n1_2024];
110.         Lagu_2511532024[] R_2024 = new
Lagu_2511532024[n2_2024];
111.
112.         for(int i_2024 = 0; i_2024 < n1_2024;
i_2024++) {
113.             L_2024[i_2024] =
dataLagu_2024[left_2024 + i_2024];
114.         }
115.
116.         for(int j_2024 = 0; j_2024 < n2_2024;
j_2024++) {
117.             R_2024[j_2024] =
dataLagu_2024[mid_2024 + 1 + j_2024];
118.         }
119.
120.         int i_2024 = 0;
121.         int j_2024 = 0;
122.         int k_2024 = left_2024;
123.
124.         while(i_2024 < n1_2024 && j_2024 <
n2_2024) {
125.             if(L_2024[i_2024].judul_2024.compareToIgnoreC
ase(
126.                 R_2024[j_2024].judul_2024) <= 0) {
127.
128.                 dataLagu_2024[k_2024++] =
L_2024[i_2024++];
129.             } else {
130.                 dataLagu_2024[k_2024++] =
R_2024[j_2024++];
131.             }
132.         }
133.
134.         while(i_2024 < n1_2024) {
135.             dataLagu_2024[k_2024++] =
L_2024[i_2024++];
136.         }
137.         while(j_2024 < n2_2024) {

```

```

138.         dataLagu_2024[k_2024++] =
R_2024[j_2024++];
139.     }
140. }
141.
142.     static void mergeSort_2024(int left_2024, int
right_2024) {
143.         if(left_2024 < right_2024) {
144.             int mid_2024 = (left_2024 +
right_2024) / 2;
145.
146.             mergeSort_2024(left_2024,
mid_2024);
147.             mergeSort_2024(mid_2024 + 1,
right_2024);
148.             merge_2024(left_2024, mid_2024,
right_2024);
149.         }
150.     }
151.
152.     public static void main(String[] args) {
153.         Scanner input_2024 = new
Scanner(System.in);
154.         inputData_2024();
155.
156.         System.out.println("=== Sorting
Playlist NIM: 2511532024 ===");
157.         System.out.print("Pilih Algoritma " +
"(1=shell, 2=quick, 3=merge): ");
158.         int pilihan_2024 = input_2024.nextInt();
159.         System.out.println("\nData Sebelum
Sorting: ");
160.
161.         tampilData_2024();
162.         switch(pilihan_2024) {
163.             case 1:
164.                 shellSort_2024();
165.                 System.out.println("\nData
Setelah Shell Sort (Judul A-Z): ");
166.                 break;
167.
168.             case 2:
169.                 quickSort_2024(0,
jumlahData_2024 - 1);
170.                 System.out.println("\nData
Setelah Quick Sort (Durasi Asc): ");
171.                 break;
172.
173.             case 3:
174.                 mergeSort_2024(0,
jumlahData_2024 - 1);
175.                 System.out.println("\nData
Setelah Merge Sort (Judul A-Z): ");
176.                 break;
177.
178.             default:

```

```

179.         System.out.println("Pilihan tidak valid");
180.
181.         return;
182.     }
183.
184.     tampilData_2024();
185. }
186.
187. }

```

4.3 Penjelasan

4.3.1 Class Lagu_2511532024

1. Deklarasi Kelas

```
class Lagu_2511532024 {
```

Syntax di atas digunakan untuk membuat class Lagu_2511532024 yang berfungsi sebagai blueprint atau cetakan objek lagu. Class ini digunakan untuk menyimpan informasi yang berkaitan dengan lagu, seperti judul, penyanyi, dan durasi lagu

2. Deklarasi Atribut

```
String judul_2024;
String penyanyi_2024;
int durasi_2024;
```

Syntax di atas digunakan untuk mendeklarasikan atribut atau variabel anggota dari class Lagu_2511532024. Variabel judul_2024 digunakan untuk menyimpan judul lagu, penyanyi_2024 digunakan untuk menyimpan nama penyanyi, dan durasi_2024 digunakan untuk menyimpan durasi lagu dalam satuan detik.

3. Constructor Lagu

```
public Lagu_2511532024(
    String judul_2024,
    String penyanyi_2024,
```

```
int durasi_2024) {
```

Syntax di atas merupakan constructor dari class Lagu_2511532024. Constructor digunakan untuk menginisialisasi objek saat objek tersebut dibuat. Nilai judul lagu, penyanyi, dan durasi lagu akan diterima melalui parameter constructor.

4. Inisialisasi Nilai Atribut

```
this.judul_2024 = judul_2024;  
this.penyanyi_2024 = penyanyi_2024;  
this.durasi_2024 = durasi_2024;
```

Syntax di atas digunakan untuk mengisi atribut objek dengan nilai yang diterima dari parameter constructor. Kata kunci this digunakan untuk merujuk pada atribut milik objek yang sedang dibuat sehingga dapat dibedakan dengan parameter yang memiliki nama sama.

4.3.2 Class Sorting_2511532024

1. Import Library dan Deklarasi Class

```
import java.util.Scanner;  
public class Sorting_2511532024 {
```

Syntax di atas digunakan untuk mengimpor library Scanner yang berfungsi menerima input dari pengguna melalui keyboard. Class Sorting_2511532024 merupakan class utama yang berisi seluruh proses pengolahan data lagu dan algoritma pengurutan.

2. Deklarasi Array dan Variabel Jumlah Data

```
static Lagu_2511532024[] dataLagu_2024 =  
    new Lagu_2511532024[20];  
static int jumlahData_2024 = 0;
```

Syntax di atas digunakan untuk membuat array objek Lagu_2511532024 yang berfungsi menyimpan data lagu. Variabel jumlahData_2024 digunakan untuk menghitung jumlah data yang telah dimasukkan ke dalam array.

3. Method Input Data Lagu

```
static void inputData_2024() {  
    ...  
}
```

Method inputData_2024() digunakan untuk mengisi data lagu ke dalam array. Setiap data lagu dibuat menggunakan constructor dari class Lagu_2511532024 dan disimpan ke dalam array dataLagu_2024.

4. Menambahkan Data Lagu ke Array

```
dataLagu_2024[jumlahData_2024++] =  
    new Lagu_2511532024(  
        "The Apartment We Won't Share", "NIKI", 150);
```

Syntax di atas digunakan untuk membuat objek lagu baru dan menyimpannya ke dalam array. Operator ++ digunakan untuk menambah nilai jumlahData_2024 setiap kali data baru berhasil dimasukkan.

5. Method Menampilkan Data

```
static void tampilData_2024() {  
    ...  
}
```

Method tampilData_2024() digunakan untuk menampilkan seluruh data lagu yang terdapat dalam array ke layar sehingga pengguna dapat melihat isi data sebelum dan sesudah proses pengurutan.

6. Perulangan Menampilkan Data

```
for(int i_2024 = 0; i_2024 < jumlahData_2024; i_2024++)  
{
```

Perulangan digunakan untuk mengakses seluruh elemen array dan menampilkan informasi lagu berupa judul serta durasi lagu.

7. Method Shell Sort

```
static void shellSort_2024() {  
    ...  
}
```

Method shellSort_2024() digunakan untuk mengurutkan data lagu berdasarkan judul secara ascending (A-Z) menggunakan algoritma Shell Sort. Algoritma ini bekerja dengan membandingkan elemen yang memiliki jarak tertentu (gap) hingga seluruh data tersusun secara terurut.

8. Method Partition

```
static int partition_2024(int low_2024, int high_2024) {
```

Method partition_2024() digunakan dalam algoritma Quick Sort untuk membagi array menjadi dua bagian berdasarkan nilai pivot. Lagu dengan durasi lebih kecil dari pivot ditempatkan di sebelah kiri, sedangkan yang lebih besar ditempatkan di sebelah kanan.

9. Method Quick Sort

```
static void quickSort_2024(int low_2024, int high_2024) {
```

Method quickSort_2024() digunakan untuk mengurutkan data lagu berdasarkan durasi secara ascending menggunakan algoritma Quick Sort. Proses pengurutan dilakukan secara rekursif dengan memanfaatkan method partition_2024().

10. Method Merge

```
static void merge_2024(int left_2024, int mid_2024,  
    int right_2024) {
```

Method `merge_2024()` digunakan untuk menggabungkan dua subarray yang telah terurut menjadi satu array yang tetap berada dalam urutan yang benar. Method ini merupakan bagian penting dari algoritma Merge Sort.

11. Method Merge Sort

```
static void mergeSort_2024(int left_2024, int right_2024) {
```

Method `mergeSort_2024()` digunakan untuk mengurutkan data lagu berdasarkan judul secara ascending menggunakan algoritma Merge Sort. Algoritma ini bekerja dengan membagi array menjadi bagian-bagian kecil, kemudian menggabungkannya kembali dalam keadaan terurut.

12. Method Main

```
public static void main(String[] args) {
```

Method `main()` merupakan titik awal eksekusi program. Semua proses utama program dijalankan melalui method ini.

13. Pembuatan Objek Scanner dan Input Pilihan

```
Scanner input_2024 = new Scanner(System.in);  
int pilihan_2024 = input_2024.nextInt();
```

Syntax di atas digunakan untuk membuat objek Scanner dan menerima input pilihan algoritma sorting dari pengguna.

14. Menampilkan Data Sebelum Sorting

```
System.out.println("\nData Sebelum Sorting:");  
tampilData_2024();
```

Syntax ini digunakan untuk menampilkan seluruh data lagu sebelum proses pengurutan dilakukan.

15. Pemilihan Algoritma Menggunakan Switch

```
switch(pilihan_2024) {
```

Struktur switch digunakan untuk menentukan algoritma sorting yang akan dijalankan berdasarkan pilihan pengguna, yaitu Shell Sort, Quick Sort, atau Merge Sort.

16. Menampilkan Data Setelah Sorting

```
tampilData_2024());
```

Setelah proses pengurutan selesai dilakukan, method tampilData_2024() dipanggil kembali untuk menampilkan hasil data yang telah terurut sesuai algoritma yang dipilih.

4.4 Output Kode Program

```
=== Sorting Playlist NIM: 2511532024 ===
Pilih Algoritma (1=shell, 2=quick, 3=merge): 3

Data Sebelum Sorting:
1. The Man Who Can't Be Moved - 241 detik
2. The Apartment We Won't Share - 150 detik
3. Best Friend - 262 detik
4. Needy - 172 detik
5. Hold Me Down - 231 detik
6. Multo - 238 detik
7. Bags - 261 detik
8. Seasons - 256 detik

Data Setelah Merge Sort (Judul A-Z):
1. Bags - 261 detik
2. Best Friend - 262 detik
3. Hold Me Down - 231 detik
4. Multo - 238 detik
5. Needy - 172 detik
6. Seasons - 256 detik
7. The Apartment We Won't Share - 150 detik
8. The Man Who Can't Be Moved - 241 detik
```

Saya memilih algoritma Merge Sort karena algoritma ini memiliki performa yang stabil dan mampu mengurutkan data dengan efisien. Merge Sort menggunakan metode divide and conquer, yaitu membagi data menjadi bagian-bagian kecil kemudian menggabungkannya kembali dalam keadaan terurut.

Selain itu, Merge Sort memiliki kompleksitas waktu $O(n \log n)$ sehingga tetap bekerja dengan baik meskipun jumlah data bertambah banyak. Oleh karena itu, algoritma ini cocok digunakan untuk

mengurutkan data lagu berdasarkan judul secara ascending (A-Z) pada penugasan ini.